

[Computer Vision]

실험 보고서

2026 년 5 월 21일



문서 번호 : CV2026_HW#3_2021041072

소 속 : 충북대학교 소프트웨어학과

이 름 : 전영우

교 수 : 최경주 교수님

목 차

1. 서론	1
2. 실험 A : 손실 함수에 따른 학습 성능 비교	
1.1 실험 목표	1
1.2 실험 세팅	2
1.3 그래프 및 시각화 결과	3
1.4 정량적 분석	3
1.5 해설 및 분석	3
1.6 결과 및 개선사항	5
3. 실험 B : 활성화 함수에 따른 학습 특성 비교	
2.1 실험 목표	6
2.2 실험 세팅	6
2.3 그래프 및 시각화 결과	7
2.4 정량적 분석	10
2.5 해설 및 분석	11
2.6 결과 및 개선사항	12
4. 실험 C : Optimizer 및 Learning Rate에 따른 수렴 특성 비교	
3.1 실험 목표	12
3.2 실험 세팅	13
3.3 그래프 및 시각화 결과	14
3.4 정량적 분석	15
3.5 해설 및 분석	15
3.6 결과 및 개선사항	18
5. 결론	18

서론

위 문서는 충북대학교 2026-1학기 컴퓨터 비전 과제#3에 대한 기술 문서입니다.
손실 함수, 활성화 함수, 최적화 알고리즘이 학습 결과에 미치는 영향을 구현과 실험을 통해
정량적으로 분석하고, 실험 결과를 바탕으로 하여금 성능을 비교 분석한 내용을 포함하고
있습니다.

실험 A

1. 실험 목표

1) 실험 목표

- MSE와 CrossEntropy가 학습 성능에 미치는 차이 분석
- 학습 곡선의 수렴 속도, 정확도, loss 안정성 비교

2) 추천 데이터셋 : Fashion-MNIST, Digits Dataset

3) 실험 조건

- 동일한 네트워크 구조 및 optimizer 하에 손실함수만 변경
- MSE Loss 사용 시 softmax를 출력층에 명시적으로 적용. CrossEntropy Loss는 내부적으로 softmax 포함

예시) # MSE Loss 사용 시

```
loss_fn = nn.MSELoss()  
outputs = torch.softmax(model(inputs), dim=1)
```

4) 분석 포인트

- MSE 사용 시 Gradient Vanishing 문제 분석
- CrossEntropy의 빠른 수렴 속도와 안정성 확인
- 최종 정확도 차이, 학습 곡선 비교

2. 실험 세팅

모델 정의:

```
class MLP(nn.Module):  
    def __init__(self, input_size, hidden1, hidden2, hidden3,  
num_classes):  
        super(MLP, self).__init__()  
  
        self.fc1 = nn.Linear(input_size, hidden1)  
        self.relu1 = nn.ReLU()  
  
        self.fc2 = nn.Linear(hidden1, hidden2)  
        self.relu2 = nn.ReLU()
```

```

self.fc3 = nn.Linear(hidden2, hidden3)
self.relu3 = nn.ReLU()

self.fc4 = nn.Linear(hidden3, num_classes)

def forward(self, x):
    x = self.relu1(self.fc1(x))
    x = self.relu2(self.fc2(x))
    x = self.relu3(self.fc3(x))
    x = self.fc4(x)
    return x

```

실험 데이터셋 :
Fashion-MNist

하이퍼파라미터

```

input_size = 784      # MNIST 이미지 크기 (28*28 픽셀)
hidden_size1 = 512    # 은닉층의 노드 수 (기본 네트워크 구조를 따름)
hidden_size2 = 256
hidden_size3 = 128
num_classes = 10      # 숫자 클래스 (0~9)

num_epochs = 50       # 학습 횟수
batch_size = 128      # 한 번에 처리하는 데이터 개수
learning_rate = 0.001 # 학습률 (너무 크면 발산, 너무 작으면 느린 학습) Adam을
                        # 사용하므로 0.001로 세팅함

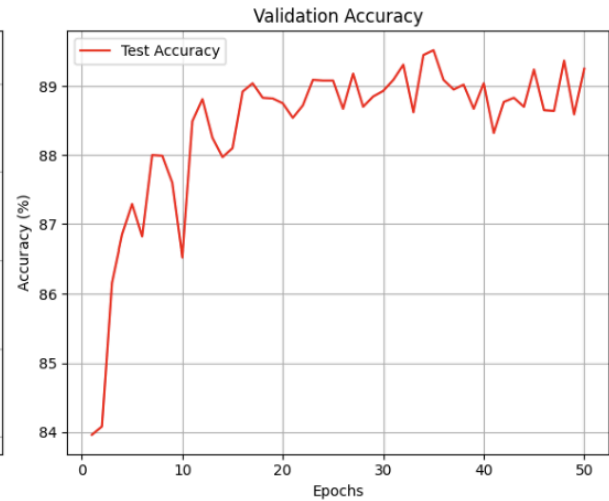
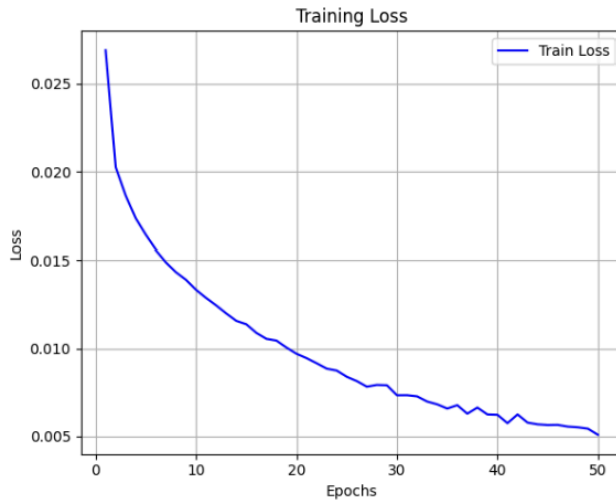
```

옵티마이저

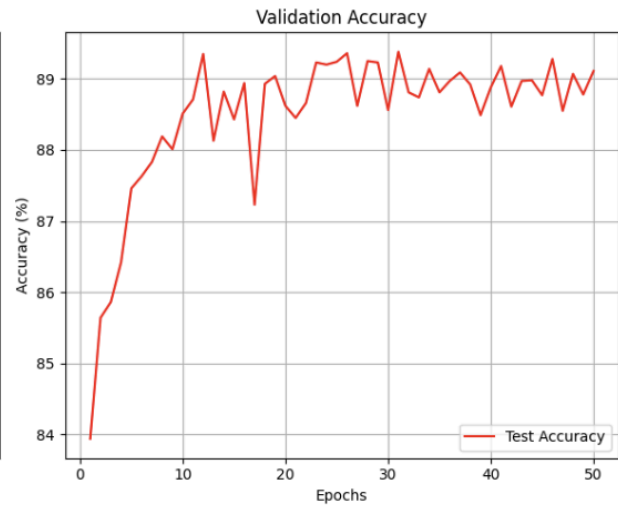
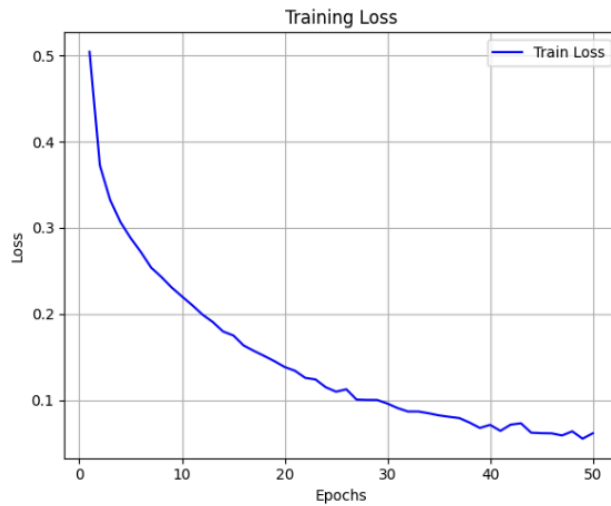
```
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

3. 그래프 및 시각화 결과

실험 A#1 (MSE + Softmax)



실험 A#2 (CrossEntropy + Softmax)



4. 정량적 분석

실험 항목	최종 정확도(%)	Loss 최솟값	수렴까지 걸린 epoch
MSE(with softmax)	89.25%	0.0051	25
CrossEntropy	89.11%	0.0614	25

5. 해설 및 분석

실험별 질문 (Cross Entropy vs MSE)

- 1) 왜 MSE는 CrossEntropy보다 학습이 느리고, 안정적이지 못한가?
 - 로지트(logits) 값의 분포 변화와 gradient scaling 문제를 설명하세요.

CrossEntropy와 MSE는 둘 다 예측값과 정답의 차이를 줄이는 방향으로 학습하지만, gradient가 전달되는 방식에서 차이점이 있습니다.

CrossEntropy Loss는 보통 softmax와 함께 사용되며, PyTorch의 CrossEntropyLoss 함수는 내부적으로 Softmax를 포함합니다. 이때 모델의 출력은 확률값이 아닌, raw logits을 가집니다.

CrossEntropy의 gradient는 단순히 $\hat{y}_i - y_i$ 의 형태로, 정답 클래스의 확률이 낮으면 강한 gradient를 발생시켜 정답 클래스의 logit은 높이고 비정답 클래스의 logit은 낮추는 방향으로 직접적인 학습 신호를 전달합니다.

반면 MSE Loss를 분류 문제에 사용하는 경우 일반적으로 softmax를 먼저 적용한 후 one-hot 정답과 비교합니다. $L_{MSE} = \sum_i (\hat{y}_i - y_i)^2$ 이러한 함수적인 특성에 기인하여 gradient는 단순히

$\hat{y}_i - y_i$ 으로 끝나지 않고 softmax의 미분값까지 곱해진 $\frac{\partial L_{MSE}}{\partial z_i} = \frac{\partial L_{MSE}}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i}$ 의 형태로 나타납니다.

따라서 MSE Loss의 gradient는 softmax의 미분값이 확률이 0 또는 1에 가까워질수록 작아진다는 특성을 가지며, softmax의 출력이 특정 클래스에 치우치기 시작하면 gradient가 매우 작아집니다. 이 때문에 MSE는 학습이 진행될수록 gradient가 줄어들고, 잘못 예측하고 있는 경우에도 충분히 강한 신호를 발생시키지 않을 수 있습니다.

결론적으로 MSE는 분류 문제에서 확률값 자체의 거리만 줄이려고 하며, softmax의 미분값이 추가로 곱해져 gradient가 작아지기 때문에 느리고 안정적이지 못한 특성을 가집니다.

2) CrossEntropy가 MSE보다 수렴 속도가 빠른 이유는 무엇인가?

- CrossEntropy가 Gradient Vanishing 문제를 어떻게 완화하는지 분석하세요.

MSE + softmax에서는 gradient에 softmax 미분값이 추가로 곱해집니다.

정답 클래스 확률이 낮은 경우에도 softmax 미분값에 의해 실제 logits에 전달되는 gradient가 작아질 수 있고, 특히 학습 후반부에는 softmax 출력이 특정 클래스에 몰리며 미분값이 작아져 gradient vanishing 문제가 더 쉽게 발생합니다.

이에 반해 Cross Entropy 함수는 정답 클래스의 예측 확률이 낮을수록 큰 loss를 전달하여금 해당 logit을 빠르게 증가시키는 방향으로 gradient를 수정합니다. 이에 따라 정답 클래스에 대한 gradient 신호가 더 직접적으로 강하게 전달됩니다.

이러한 함수적 차이점에 기인하여, 결론적으로 CrossEntropy는 Classification에서 정답 클래스의 logit을 상승시키고 비정답 클래스의 logit을 감소시켜 더 강하고 안정적인 gradient를 제공합니다.

따라서, CrossEntropy는 MSE보다 빠른 수렴 속도를 가집니다.

3) 학습 초기 vs 후반의 gradient 분포 차이

- 학습 초기에는 gradient가 크지만 후반에는 0에 수렴하는지 시각화하세요.

학습 초기에는 모델의 가중치가 충분히 학습되지 않았기에, 각 클래스의 **logits**값이 비슷하게 계산됩니다. 따라서 **softmax**의 결과도 특정 클래스에 치우치지 않고 균등한 확률 분포를 가집니다.

반면, 학습이 진행될수록 모델은 정답 클래스의 **logit**은 높이고 비정답 클래스의 **logit**은 감소시킵니다. 이로 인해 **softmax**의 출력은 점점 하나의 클래스에 집중됩니다.

CrossEntropy의 경우, 예측이 맞으면 **gradient**가 자연스럽게 작아지는 특성을 가지고 있기 때문에 분류 문제에서 이러한 학습 진행에 따른 **loss** 감소가 뚜렷하게 나타납니다.

4) MSE 사용 시 softmax를 적용하지 않으면 학습이 왜 어려운가?

MSE Loss는 연속적인 수치값의 차이를 줄이는 데 적합한 손실 함수입니다. 예측값과 실제값의 차이를 제공하여금 줄이는 이러한 방식은 회귀 문제에서 효과적입니다.

Fashion-MNIST 데이터셋 등을 타겟으로 한 분류 문제에서는, 모델의 출력이 각 클래스에 대한 확률처럼 해석될 필요가 있습니다. 다만, **MSE**를 사용할 시 **softmax**를 적용하지 않으면 모델의 출력은 확률이 아닌 **raw logits**입니다.

따라서 **Softmax**를 사용하지 않은 **MSE**를 이용하여 정답 예측을 수행할 경우, 모델은 “정답 클래스의 **logits**을 가장 크게 만들기”보단 “모델 출력값 자체를 정답 **one-hot** 벡터에 맞추기”를 타겟으로 학습을 진행하게 됩니다.

즉, **MSE**에 **softmax**를 적용하지 않으면 타겟을 “**logits**를 **one-hot labels**와 직접 비교” 하는 형태로 학습을 진행하므로, 분류 목적과 손실 함수의 목표가 어긋나 학습이 불안정해질 수 있습니다.

6. 결론 및 개선사항

이번 실험을 통해 **CrossEntropy**와 **MSE** 등 손실 함수가 학습에 영향을 미치는 영향을 분석해 보았습니다.

다만 **MSE**와 **CrossEntropy**를 사용했을 때 **CrossEntropy**가 더 안정적으로 학습되어야 하는 이론과는 달리, **learning_rate**를 0.001의 낮은 값으로 설정하는 등의 요인에 의해서 비슷한 학습분포를 보였습니다.

다른 변인들을 통제하여금 **Cross Entropy** 및 **MSE**를 사용한 경우의 특징이 도드라지게 나타나는 방법을 찾아 추가적인 실험을 진행할 필요를 느꼈습니다.

더 자세한 내용은 결론 - 공통질문 1에 기재해 두었습니다.

실험 B

1. 실험 목표

1) 실험 목표

- ReLU, LeakyReLU, Sigmoid가 학습에 미치는 영향을 분석
- Dead ReLU 발생 유도 및 LeakyReLU의 완화 효과 확인

2) 추천 데이터셋 : make_moons, make_circles

3) 실험 조건

- 동일 네트워크 및 손실함수(CrossEntropy), Optimizer(Adam 또는 SGD)
- weight 초기값을 std=0.01 등 작게 설정하여 dead neuron 상황 유도

4) 분석 포인트

- Layer별 출력값 시각화 (matplotlib 사용)
- Sigmoid는 vanishing gradient 발생 → 학습 정체 확인
- Dead ReLU 발생 시 어떤 지표가 정체되는지 분석
- Dead ReLU가 발생한 neuron 비율 측정 → 히트맵 시각화
- LeakyReLU가 Dead ReLU 문제를 얼마나 완화하는지 시각적 분석

2. 실험 세팅

모델 정의:

```
class DeepMLP(nn.Module):
    def __init__(self, input_size, hidden1, hidden2, hidden3,
num_classes):
        super(DeepMLP, self).__init__()

        self.fc1 = nn.Linear(input_size, hidden1)
        self.relu = nn.ReLU() # 해당 위치를 leaky relu, sigmoid로 변경

        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, hidden3)
        self.fc4 = nn.Linear(hidden3, num_classes)

    def forward(self, x):
        self.l1 = self.relu(self.fc1(x))
        self.l2 = self.relu(self.fc2(self.l1))
        self.l3 = self.relu(self.fc3(self.l2))

        # 추후 레이어별 기여도를 뽑기 위함. activation값 * gradient를 계산하여금
        기여도를 계산.
        if self.l1.requires_grad:
```

```

        self.l1.retain_grad()
    if self.l2.requires_grad:
        self.l2.retain_grad()
    if self.l3.requires_grad:
        self.l3.retain_grad()

    x = self.fc4(self.l3)
    return x

```

실험 데이터셋 :

make_moons

하이퍼패러미터

```

input_size = 2          # [x, y] 좌표이므로 2
num_classes = 2         # 이진 분류이므로 2
hidden_size1 = 512      # 첫 번째 은닉층의 뉴런 수
hidden_size2 = 256      # 두 번째 은닉층의 뉴런 수
hidden_size3 = 128      # 세 번째 은닉층의 뉴런 수
batch_size = 128        # 한 번에 처리하는 데이터의 개수
num_epochs = 200
learning_rate = 0.1     # 일부러 큰 lr을 주어 Dead ReLU 유도

```

옵티마이저

```

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

```

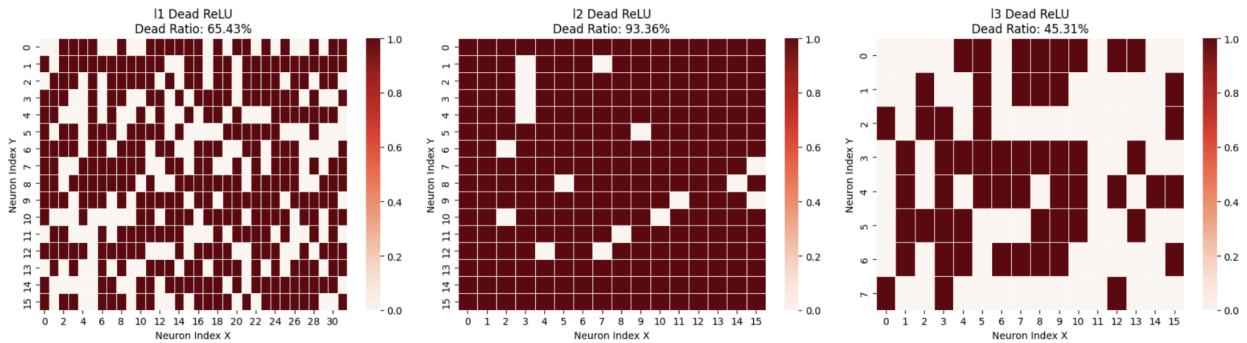
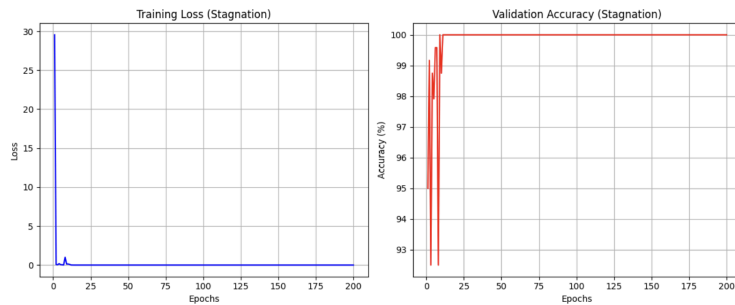
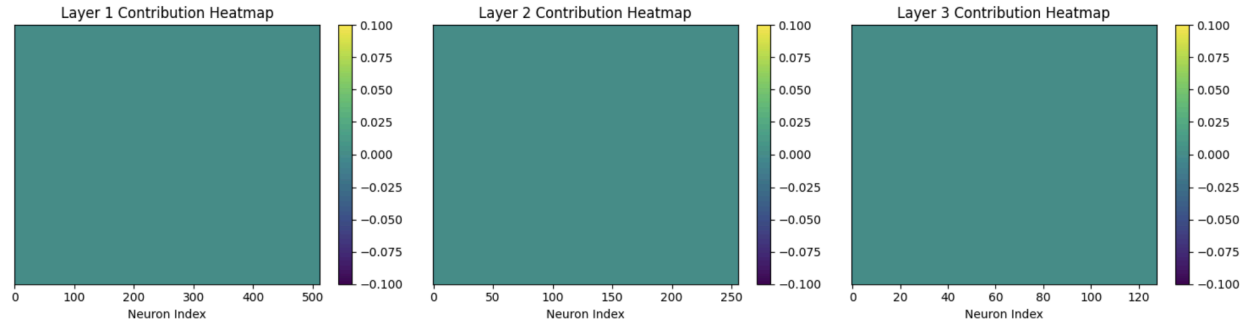
기여도 측정

activation * gradient

3. 그래프 및 시각화 결과

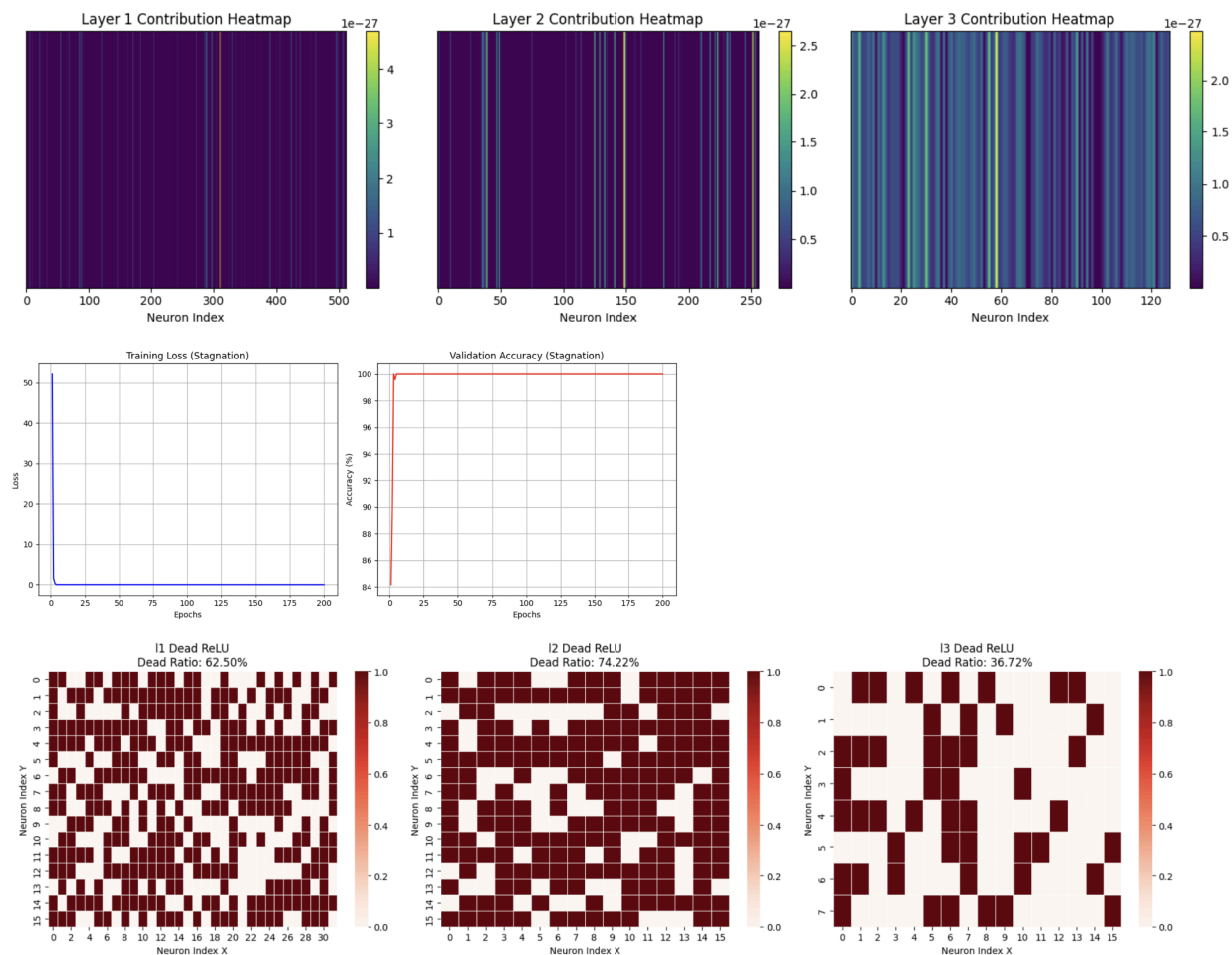
실험B#1 (relu)

Epoch [1/200], Loss: 29.5415, Accuracy: 95.00%, C1: 0.001808, C2: 0.004813, C3: 0.013986
 early activation saved at epoch 1
 Epoch [2/200], Loss: 0.0737, Accuracy: 99.17%, C1: 0.000029, C2: 0.000027, C3: 0.000025
 Epoch [3/200], Loss: 0.0399, Accuracy: 92.50%, C1: 0.000021, C2: 0.000022, C3: 0.000021
 Epoch [4/200], Loss: 0.1784, Accuracy: 88.75%, C1: 0.000063, C2: 0.000072, C3: 0.000184
 Epoch [5/200], Loss: 0.0565, Accuracy: 97.92%, C1: 0.000023, C2: 0.000023, C3: 0.000050
 Epoch [6/200], Loss: 0.0361, Accuracy: 99.58%, C1: 0.000017, C2: 0.000012, C3: 0.000030
 Epoch [7/200], Loss: 0.0128, Accuracy: 99.58%, C1: 0.000004, C2: 0.000004, C3: 0.000008
 Epoch [8/200], Loss: 1.0017, Accuracy: 92.50%, C1: 0.000192, C2: 0.000255, C3: 0.000689
 Epoch [9/200], Loss: 0.1291, Accuracy: 100.00%, C1: 0.000028, C2: 0.000045, C3: 0.000208
 Epoch [10/200], Loss: 0.1391, Accuracy: 98.75%, C1: 0.000058, C2: 0.000055, C3: 0.000128
 Epoch [11/200], Loss: 0.0861, Accuracy: 100.00%, C1: 0.000025, C2: 0.000032, C3: 0.000034



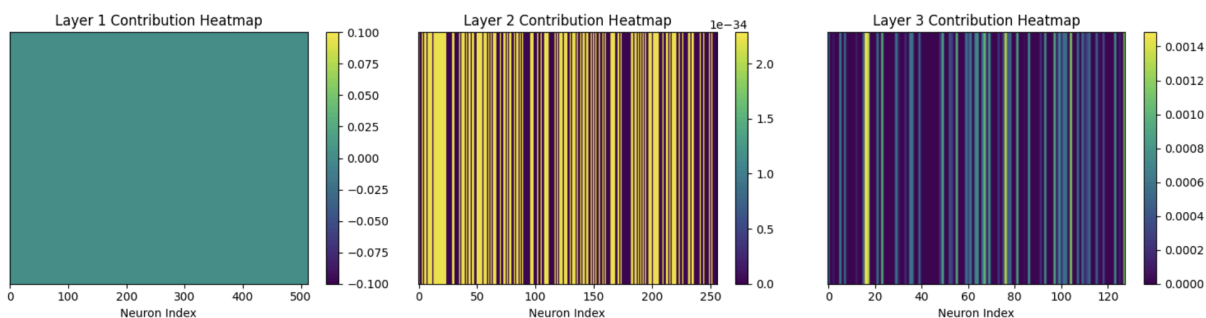
실험 B#2 (leaky relu)

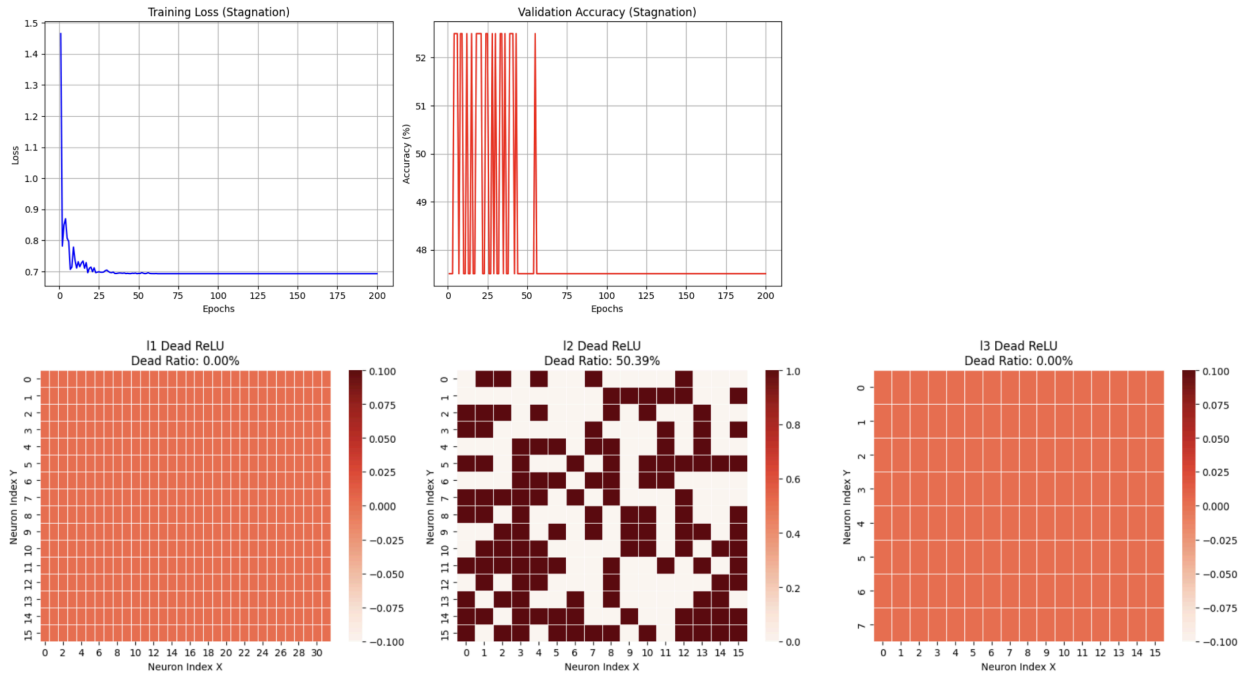
Epoch [1/200], Loss: 52.1660, Accuracy: 84.17%, C1: 0.004905, C2: 0.011733, C3: 0.026296
 Epoch [2/200], Loss: 1.6155, Accuracy: 90.00%, C1: 0.000489, C2: 0.001106, C3: 0.008365
 Epoch [3/200], Loss: 0.3363, Accuracy: 100.00%, C1: 0.000136, C2: 0.000267, C3: 0.001561
 Epoch [4/200], Loss: 0.0087, Accuracy: 99.58%, C1: 0.000015, C2: 0.000018, C3: 0.000063
 Epoch [5/200], Loss: 0.0087, Accuracy: 100.00%, C1: 0.000010, C2: 0.000012, C3: 0.000049
 Epoch [6/200], Loss: 0.0018, Accuracy: 100.00%, C1: 0.000003, C2: 0.000004, C3: 0.000015



실험 B#3 (Sigmoid)

Epoch [55/200], Loss: 0.6941, Accuracy: 52.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [56/200], Loss: 0.6962, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [57/200], Loss: 0.6939, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [58/200], Loss: 0.6934, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [59/200], Loss: 0.6933, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [60/200], Loss: 0.6933, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [61/200], Loss: 0.6936, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [62/200], Loss: 0.6931, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247
 Epoch [63/200], Loss: 0.6932, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247
 Epoch [64/200], Loss: 0.6931, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247
 Epoch [65/200], Loss: 0.6931, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247
 Epoch [66/200], Loss: 0.6932, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [67/200], Loss: 0.6931, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000248
 Epoch [68/200], Loss: 0.6933, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247
 Epoch [69/200], Loss: 0.6931, Accuracy: 47.50%, C1: 0.000000, C2: 0.000000, C3: 0.000247





4. 정량적 분석

활성화 함수	Dead ReLU비율(%)		최종 정확도(%)	수렴속도
ReLU	I1	65.43%	100	빠름
	I2	93.36%		
	I3	45.31%		
LeakyReLU	I1	62.50%	100	매우 빠름
	I2	74.22%		
	I3	36.72%		
Sigmoid	I1	0.00%	47.50%	매우 느림
	I2	50.39%		
	I3	0.00%		

이번 실험에서는 **Dead neuron** 상황 유도를 위해 고의적으로 0.1의 높은 **learning rate**를 부여하여 실험을 진행했습니다.

ReLU 함수를 이용하여 트레이닝을 진행 (**B#1**)한 결과 약 10번의 **epoch**가 지나면서, 이후부터 100%의 **Accuracy**가 나왔습니다.

Leaky Relu 함수를 이용하여 트레이닝을 진행(B#2)하였을 때는, **Relu** 함수를 이용했던 것보다 빠른 6번의 Epoch 이후부터 100%의 **Accuracy**가 나왔습니다.

이에 반해 **Cross Entropy + Softmax**를 이용하여 실험(B#3)을 진행했을 때는, **loss** 및 **Accuracy**가 수렴하지 않고 계속해서 진동하며, 학습이 진행되지 않다가, 약 65번 epoch가 지난 시점부터 낮은 **Accuracy**에 수렴하여 학습이 진행되지 않는 것을 확인할 수 있었습니다.

5. 해설 및 분석

실험별 질문 (활성함수 따른 학습 결과 비교)

1) Dead ReLU가 발생한 비율이 높으면 학습에 미치는 영향은?

- 특정 Layer에서 Dead ReLU 비율이 높으면 학습이 정체되는 이유를 분석하세요.

Dead ReLU 비율이 높다는 것은 많은 뉴런이 계속해서 0만 출력하고 있다는 의미입니다. 이러한 과정에서 발생한 **dead neuron**은 역전파 과정에서 **weight update**를 거의 받지 못하며, 학습에 기여하지 못합니다.

특정 layer에서 **dead neuron** 비율이 높아지면 다음 layer로 전달되는 정보량이 줄어들며, 그 결과 모델의 표현력이 감소하고 **decision boundary**를 충분히 학습하지 못하게 됩니다.

특히 깊은 layer에서 **dead neuron** 이 많으면 앞 layer에서 추출한 특징들이 제대로 조합되지 못하고, 이로 인해 **loss**감소가 느려져 **accuracy** 상승이 정체됩니다.

2) LeakyReLU가 Dead ReLU를 얼마나 완화하는가?

- LeakyReLU 사용 시 Dead 상태에 빠진 뉴런이 감소하는지 확인하세요.

ReLU 함수는 $f(x) = \max(0, x)$ 이며, **Leaky ReLU**는 $f(x) = \max(\alpha x, x)$ 입니다.

여기서 α 는 보통 0.01 정도의 작은 값이며, 이에 따라 입력이 음수여도 출력이 완전히 0이 되지 않고 **gradient** 또한 완전히 사라지지 않습니다. 따라서 **neuron**이 음수 영역에 있더라도 다시 **weight update**를 통해 다시 활성화될 가능성이 있습니다.

실험 결과에서도 이러한 경향성을 확인할 수 있습니다. **ReLU** 함수에서는 약 65%, 93%, 45%의 **dead neuron** 비율이 발생한 반면 **Leaky ReLU** 함수를 활용한 경우엔 약 62.5%, 74.22%, 36.72%로 **ReLU**에 비해 낮은 **dead neuron** 비율을 보였습니다.

3) Sigmoid를 사용할 때 vanishing gradient가 발생하는 구간은 어디인가?

- Sigmoid의 gradient가 0에 가까워지는 지점을 시각화하고 설명하세요.

Sigmoid 함수의 **gradient**는 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ 입니다.

Sigmoid 출력이 0.5일 때 **gradient**는 최대값 0.25를 가집니다. 다만, 출력이 0 또는 1에 가까워질수록 **gradient**는 0에 가까워집니다.

sigmoid activation histogram에서 값이 0 또는 1 근처에 집중되어 분포한다면, 해당 layer에서는 gradient vanishing이 발생하고 있다고 볼 수 있습니다.

위의 실험 결과에서 나타난 학습 추이를 보면 Sigmoid 함수를 활용한 경우, 약 50 epoch가 지난 시점에서부터 accuracy 업데이트가 이루어지지 않고 있음을 확인할 수 있습니다. 이로 미루어 볼 때, 해당 지점에서부터 vanishing gradient 문제가 발생했음을 시각적으로 확인할 수 있었습니다.

4) 각 Layer가 학습에 기여하는 정도를 히트맵으로 분석하세요.

저는 레이어가 학습에 기여하는 정도를 $\text{activation} * \text{gradient}$ 로 정의하여금 해당 layer의 뉴런이 loss 변화에 얼마나 민감하게 관여하는지 확인했습니다. ReLU, Leaky ReLU, Sigmoid의 레이어별 기여도 히트맵을 비교한 결과, 활성화함수에 따라 뉴런의 학습 기여 양상이 다르게 나타남을 확인할 수 있었습니다.

ReLU는 전체 레이어에서 기여도 값이 거의 0에 가까운 균일한 분포를 보였으며, 이는 일부 뉴런이 비활성화되어 학습에 충분히 기여하지 못했을 가능성이 있음을 나타냅니다.

반면 Leaky ReLU는 레이어 별로 더 뚜렷한 기여도 분포를 보였으며, 음수 영역에서도 gradient가 유지되어 ReLU보다 더 많은 뉴런이 학습에 참여한 것으로 해석할 수 있습니다.

Sigmoid는 Layer 2와 Layer 3에서 일부 뉴런 중심의 강한 기여도가 나타났지만, 이는 전체 학습 효율이 높다는 의미보다는 특정 뉴런에 기여가 편중되었다고 해석합니다.

6. 결론 및 개선사항 : 결론 - 공통질문 1에 기재

실험 C

1. 실험 목표

1) 실험 목표

- SGD, SGD+Momentum, Adam의 성능 비교
- 학습률 변화가 미치는 영향 분석 (0.1, 0.01, 0.001)

2) 추천 데이터셋: Fashion-MNIST, Digits Dataset

3) 실험 조건

- 동일한 네트워크, 손실함수, 데이터셋
- 학습률에 따른 3개의 optimizer 성능 비교 (표로 정리)
- Learning rate를 지수 감소 (Exponential Decay) 적용하여 실험

```
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
```

4) 분석 포인트

- Overshooting, 느린 수렴, 안정성 확인
- 학습률 변화에 따른 Loss 곡선과 Gradient 흐름 분석

2. 실험 세팅

모델 정의:

```
class MLP(nn.Module):
    def __init__(self, input_size, hidden1, hidden2, hidden3,
num_classes):
        super(MLP, self).__init__()

        self.fc1 = nn.Linear(input_size, hidden1)
        self.relu1 = nn.ReLU()

        self.fc2 = nn.Linear(hidden1, hidden2)
        self.relu2 = nn.ReLU()

        self.fc3 = nn.Linear(hidden2, hidden3)
        self.relu3 = nn.ReLU()

        self.fc4 = nn.Linear(hidden3, num_classes)

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.relu3(self.fc3(x))
        x = self.fc4(x)
        return x

model = MLP(input_size, hidden_size1, hidden_size2, hidden_size3,
num_classes).to(device)
```

실험 데이터셋 :
Fashion-MNist

하이퍼파라미터

하이퍼 파라미터 설정 : 모델의 구조와 학습 조건을 정의하는 핵심 변수들.

```

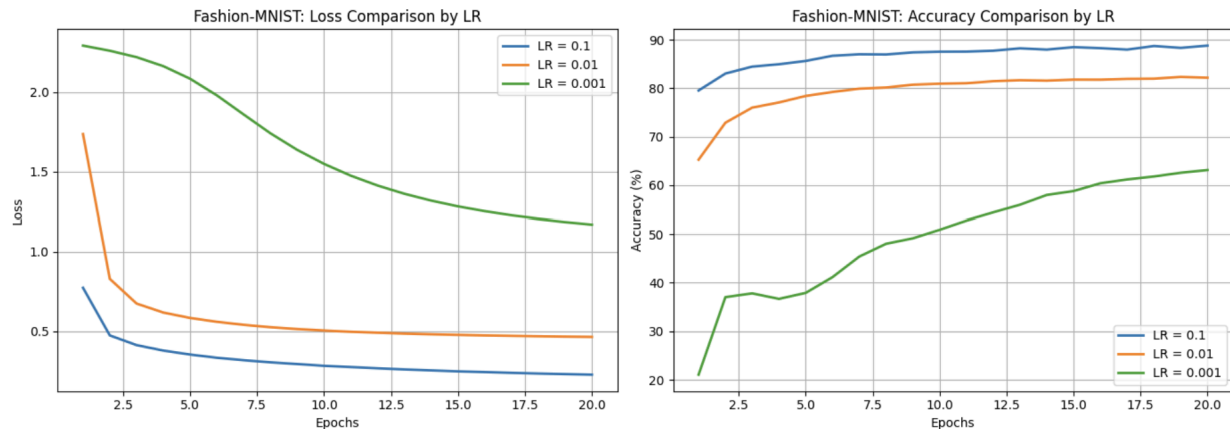
input_size = 784      # MNIST 이미지 크기 (28*28 픽셀)
input_size = 784      # MNIST 이미지 크기 (28*28 픽셀)
hidden_size1 = 512    # 은닉층의 노드 수
hidden_size2 = 256
hidden_size3 = 128
num_classes = 10      # 숫자 클래스 (0~9)
num_classes = 10      # 숫자 클래스 (0~9)
num_epochs = 20       # 학습 횟수
batch_size = 128      # 한 번에 처리하는 데이터 개수
lr_list = [0.1, 0.01, 0.001] # 학습률 (너무 크면 발산, 너무 작으면 느린 학습)
lr_results = {}

```

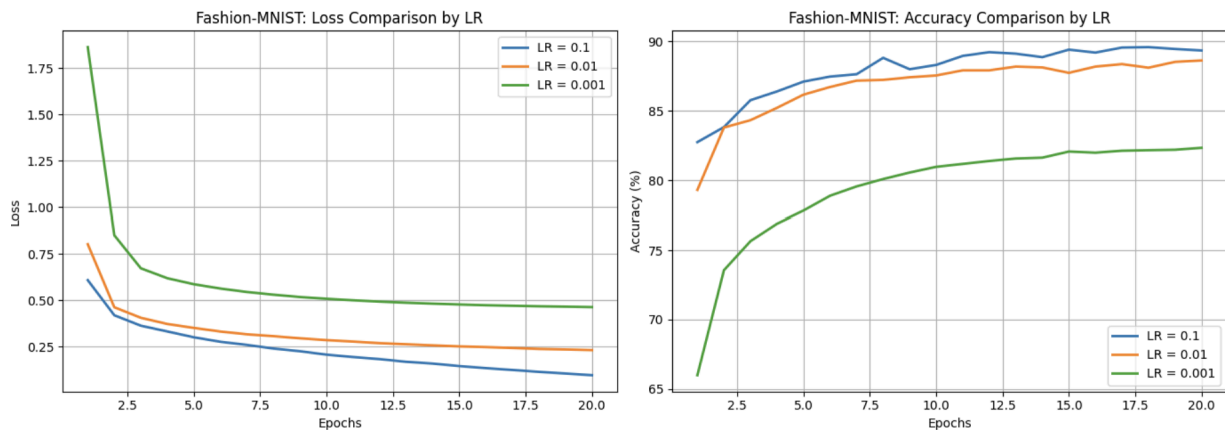
손실함수:

```
loss_fn = nn.CrossEntropyLoss()
```

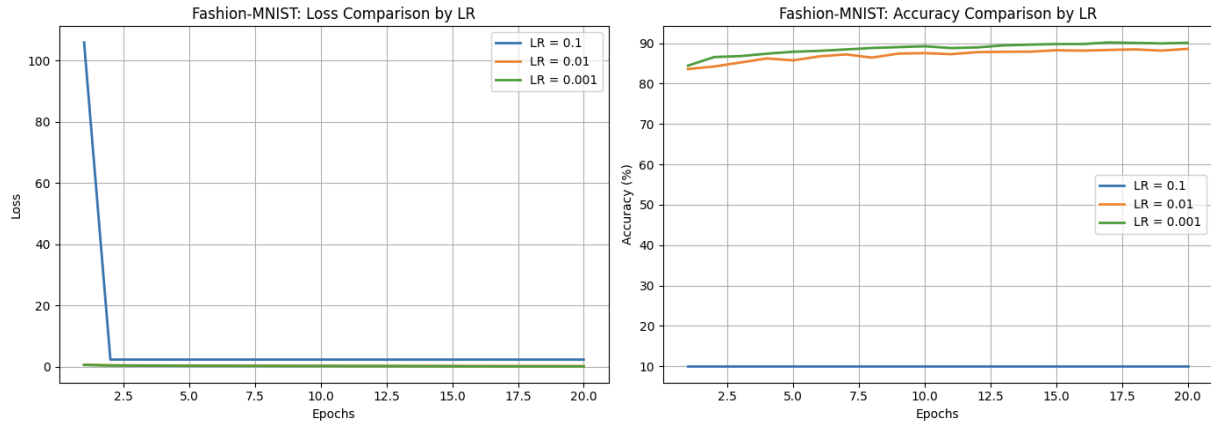
3. 그래프 및 시각화 결과 실험C#1 (SGD)



실험C#2 (SGD + Momentum)



실험C#3 (Adam)



4. 정량적 분석

Optimizer	Learning Rate	최종 정확도(%)	수렴 속도	안정성
SGD	0.1	88.78	빠름	다소 불안정
	0.01	82.16	중간	안정적
	0.001	63.12	느림	안정적
SGD + Momentum	0.1	89.34	빠름	안정적
	0.01	88.62	빠름	안정적
	0.001	82.33	느림	안정적
Adam	0.1	10.00	빠름	불안정
	0.01	88.60	빠름	안정적
	0.001	90.10	빠름	안정적

5. 해설 및 분석

실험별 질문 (Optimizer 및 Learning Rate에 따른 수렴 특성 비교)

- 1) SGD, SGD+Momentum, Adam의 학습 곡선이 다른 이유는 무엇인가?
- SGD+Momentum이 왜 SGD보다 빠른 수렴을 보여주는지 설명하세요.

기본적인 **SGD**는 현재 미니배치에서 계산된 **gradient**만을 이용해서 파라미터 업데이트를 진행(수식 : $w_{t+1} = w_t - \eta \nabla L(w_t)$)합니다. **SGD**는 구조가 단순하지만, 매 **step**마다 현재의 **gradient**만 반영하므로 **loss**곡선이 흔들리기 쉽고, 수렴 속도가 다소 느릴 수 있습니다.

반면 **SGD**에 **Momentum**을 적용하면 이전 **step**들의 이동 방향을 누적하여 반영할 수 있습니다.

(수식: $v_t = \mu v_{t-1} - \eta \nabla L(w_t)$, $w_{t+1} = w_t + v_t$) Momentum은 이전 업데이트 방향을 기억하기 때문에, 같은 방향으로 이동하는 구간에서는 속도가 증가하는 특성을 가집니다. 따라서 단순한 SGD보다 더 빠르게 loss가 감소하는 경향을 보이며, 이전 방향성이 반영되기 때문에 학습 곡선이 상대적으로 안정적이라는 특징을 가집니다.

2) 학습률 변화가 학습 안정성에 미치는 영향은?

- 너무 큰 학습률은 overshooting, 너무 작은 학습률은 정체를 초래함을 분석하세요.

학습률은 한 번의 업데이트에서 파라미터를 얼마나 크게 이동시킬지 결정하는 값입니다.

학습률이 너무 큰 경우에는 최적점 근처에 도달하더라도 한 번의 업데이트 폭이 너무 커서 최적점을 지나쳐버릴 수 있습니다. 이러한 문제점을 overshooting이라고 합니다.

반면 학습률이 너무 작은 경우에는 업데이트 폭이 지나치게 작아지며, 이러한 경우 최적점 방향으로 이동하지만 속도가 매우 느립니다. 따라서 loss가 거의 줄어들지 않거나 accuracy가 초반부터 정체되는 것처럼 보일 수 있습니다.

3) Adam이 초기 학습에 빠르게 수렴하는 이유는 무엇인가?

- 적응형 학습률(Adaptive Learning Rate) 덕분에 초기에 빠르게 최적점으로 수렴함을 설명하세요.

Adam이 초기 학습에서 빠르게 수렴하는 이유는 파라미터별로 학습률을 자동 조정하는 구조를 가지고 있기 때문입니다.

SGD는 모든 파라미터에 동일한 learning rate를 적용합니다. 하지만 실제 신경망에서는 어떤 파라미터는 gradient가 크고, 어떤 파라미터는 gradient가 작습니다. 모든 파라미터에 같은 크기의 학습률을 적용하면 일부 파라미터는 너무 크게 업데이트되고, 일부 파라미터는 너무 느리게 업데이트될 수 있습니다.

Adam은 gradient의 1차 모멘트와 2차 모멘트를 사용합니다. ($m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$) 여기서 m_t 는 gradient의 이동 평균이고, v_t 는 gradient 제곱의 이동 평균입니다. Adam은 이 값을 이용해 파라미터의 업데이트 크기를 조절합니다.

gradient가 큰 파라미터는 업데이트 크기를 줄이고 gradient가 작은 파라미터는 상대적으로 업데이트가 잘 이루어지도록 조정하며, 이를 통해 Adam은 초기에 빠르게 loss를 감소시켜 최적점으로 수렴하는 경향이 있습니다.

4) 지수 감소(Exponential Decay)를 사용했을 때 성능 변화는?

- 학습 후반부에 과적합이 줄어들거나 수렴 안정성이 증가하는지 분석하세요..

지수 감소는 epoch이 진행될수록 학습률을 일정 비율로 감소시키는 방식입니다.

이번 실험에서는 scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)와 같은 형태로 지수 감소를 정의하여 사용했습니다.

이 경우, 매 **epoch**마다 학습률이 이전 값의 **0.9**배가 됩니다.

초기에는 비교적 큰 학습률로 빠르게 최적점 근처까지 이동하고, 후반에는 학습률이 점점 작아지며 세밀한 조정이 이루어집니다. 따라서 지수 감소를 적용하면 학습 후반부의 **loss** 진동이 줄어들고, 수렴이 안정적으로 이루어질 수 있습니다.

학습률이 큰 경우에는 초반에는 빠르게 학습되지만 후반에 **overshooting**이 발생할 수 있습니다. 이때 지수 감소를 적용하면 학습이 진행될수록 업데이트 폭이 줄어들므로 최적점 주변에서 더 안정적으로 수렴할 수 있습니다.

다만 학습률이 너무 빨리 감소하면 충분히 학습하지 못했음에도 업데이트가 작아져서 학습이 정체될 수 있으므로, 적절한 **gamma**값 설정이 중요합니다.

5) Gradient 흐름이 특정 Optimizer에서 사라지거나 급격히 변할 때, 어떤 문제가 발생하는가?
- Vanishing Gradients, Exploding Gradients의 원인 분석

Gradient 흐름은 역전파 과정에서 손실함수의 변화량이 각 층의 가중치로 얼마나 잘 전달되는지를 의미합니다.

Vanishing Gradient는 **Gradient**가 매우 작아지는 경우를 말합니다. 이 경우 앞쪽 층의 가중치가 거의 업데이트 되지 않습니다. 반면 **Exploding Gradient**는 **gradient**가 지나치게 커져, 업데이트 한 번의 보폭이 너무 커져 **loss**가 급격히 증가하거나 **NaN**이 발생하는 현상을 의미하며, 학습률이 큰 경우 이러한 문제가 발생할 수 있습니다.

이렇게 **Gradient**가 너무 작아지거나 커지면 학습에 문제가 발생할 수 있습니다. 위의 실험 **C#3**의 결과를 보면 **epoch**이 진행됨에도 불구하고 **Loss**가 **0**으로 수렴하여 **Accuracy** 업데이트되고 있지 않습니다. 이를 통해 시각적으로 **vanishing gradient** 문제를 확인할 수 있었습니다.

6) Optimizer가 동일한 네트워크에서 서로 다른 학습 패턴을 보이는 이유는?
- SGD, Adam, SGD+Momentum의 학습 곡선이 왜 다르게 움직이는지, 수식적 근거를 포함하여 설명하세요

동일한 네트워크와 데이터셋을 사용하더라도 **optimizer**가 다르면 파라미터의 업데이트 방식이 달라지기 때문에 학습 패턴이 다르게 나타납니다.

SGD는 다음과 같이 현재의 **gradient**만 사용합니다. (수식 : $w_{t+1} = w_t - \eta g_t$)
따라서 **gradient**가 **noisy**하면 **loss** 곡선도 흔들립니다.

SGD + Momentum의 경우, 이전 **update** 정보를 함께 사용합니다. (수식 : $v_t = \beta v_{t-1} + \eta g_t$
 $w_{t+1} = w_t - v_t$) 이러한 수식적인 특성에 기인하여, 수렴 방향이 일정한 경우 단순히 **SGD**만 이용하는 것보다 더 빠르게 이동할 수 있습니다.

Adam의 경우 gradient의 평균과 제곱 평균을 모두 사용합니다.

(수식: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ $w_{t+1} = w_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$)

따라서 Adam은 파라미터별로 업데이트 크기를 다르게 조절할 수 있습니다.

이러한 차이 때문에 같은 모델을 사용하더라도 SGD는 느리고 흔들리는 곡선을, Momentum은 더 빠르고 안정적인 곡선을, Adam은 초기 수렴이 빠른 곡선을 보이게 됩니다.

6. 결론 및 개선사항 : 결론 - 공통질문 1에 기재

결론

공통질문

- 손실 함수, 활성화 함수, 최적화 알고리즘이 학습 곡선에 미치는 영향은 무엇인가?
- 수렴 속도, 진동 발생 여부, 학습 정체 구간을 그래프로 설명하세요.

1. 손실 함수의 영향 (MSE vs CrossEntropy)

이론적으로는 CrossEntropy Loss를 사용하는 경우 초반부터 loss가 빠르게 감소하며 accuracy가 안정적으로 상승하는 반면, MSE Loss를 사용한 모델은 loss 감소 속도가 상대적으로 느리고 accuracy 상승도 더디게 나타나야 합니다.

이는 MSE가 분류 문제에서 softmax 출력값과 one-hot label 사이의 단순 오차를 줄이는 방식이기 때문에, softmax 출력이 포화될 경우 gradient가 작아져 학습이 느려질 수 있기 때문입니다. 따라서 다중 클래스 분류 문제에서는 일반적으로 CrossEntropy가 MSE보다 수렴 속도 및 안정성 측면에서 더 적합합니다.

다만 이번 실험에서는 learning rate를 낮은 값인 0.001로 설정하여, 이러한 MSE를 사용했을 때 발생하는 수렴 속도의 불안정성은 확인할 수 없었습니다.

2. 활성화 함수의 영향 (ReLU, LeakyReLU, Sigmoid)

실험 B 결과, ReLU와 LeakyReLU는 비교적 빠르게 loss가 감소하고 Accuracy가 상승했습니다. LeakyReLU는 음수 영역에서도 작은 gradient를 유지함으로써 ReLU에서 발생할 수 있는 dead neuron 문제를 완화했습니다.

반면, Sigmoid의 경우 일정 시점 이후 loss와 accuracy가 거의 변하지 않는 학습 정체 구간이 나타났습니다. 이는 Sigmoid 함수의 출력이 0 또는 1에 가까워지면서 gradient가 매우 작아지고, 깊은 층을 거치는 과정에서 gradient가 소실되었기 때문으로 해석할 수 있습니다.

c,f) Dead ReLU의 영향과 해소 방법

Dead ReLU는 ReLU 뉴런의 출력이 계속 0으로 고정되어 더 이상 학습에 기여하지 못하는 현상입니다. 이러한 현상은 ReLU의 수식이 $\max(0, x)$ 임에 기인하며, 어떤 뉴런이 계속 음수 입력만 받는 경우 출력값과 gradient가 0이 되어 weight가 업데이트 되지 않습니다.

이러한 **Dead ReLU**를 해결하는 방법은 다음과 같습니다.

1) 학습률 낮추기

학습률이 너무 크면 weight가 한 번에 크게 이동하여 ReLU 입력값이 음수 영역으로 밀려날 수 있고, 이에 따라 해당 뉴런의 출력이 지속적으로 0이 되고 gradient 또한 전달되지 않아 Dead ReLU가 발생할 수 있습니다.

따라서 learning rate를 낮추면 가중치 갱신 폭을 낮춰 Dead ReLU 발생 가능성을 완화할 수 있습니다.

2) LeakyReLU 사용하기

ReLU 대신 LeakyReLU를 쓰면 음수 영역에서도 아주 작은 기울기를 남깁니다. 이를 통해 gradient가 완전히 차단되는 현상을 방지할 수 있고, 일부 뉴런이 음수 영역에 머물더라도 weight update를 계속 가능하게 하여 Dead ReLU 문제를 완화할 수 있습니다.

3) Batch Normalization 사용하기

Batch Normalization을 사용하면 각 layer의 입력 분포를 안정화하여 ReLU 입력값이 지나치게 음수로 치우치는 현상을 방지할 수 있습니다.

4) Bias를 작은 양수로 초기화하기

bias를 작은 양수로 초기화하면 학습 초기에 ReLU 입력값이 음수 영역으로 치우치는 것을 줄일 수 있습니다. 이를 통해 뉴런이 처음부터 비활성화 되는 가능성을 낮출 수 있습니다.

```
ex) nn.init.constant_(self.fc1.bias, 0.01)
    nn.init.constant_(self.fc2.bias, 0.01)
    nn.init.constant_(self.fc3.bias, 0.01)
```

3. 최적화 알고리즘의 영향 (SGD, SGD+Momentum, Adam)

실험 C 결과, SGD는 학습률이 작은 경우 loss 감소가 느리고 accuracy 상승도 더디게 나타났습니다.

SGD + Momentum은 SGD보다 빠르게 이동할 수 있지만, learning rate가 큰 경우 이전 gradient 방향이 누적되면서 loss 곡선에 진동이 나타날 수 있으며, 이러한 경향성을 실험 C#2의 learning rate가 0.1인 경우 확인할 수 있었습니다.

Adam의 경우 각 파라미터별로 업데이트 크기를 조절하기 때문에 초반 수렴 속도가 빠르고 비교적 안정적인 학습 곡선을 보였습니다. 다만, 학습률을 0.1인 큰 값으로 설정한 경우 학습의

보폭이 큼에 따라서 **Accuracy** 및 **Loss**가 더이상 업데이트 되지 않고 정체되어 있음을 확인할 수 있었습니다.

- 2) Loss 감소와 Accuracy 상승 간의 불균형이 발생한 경우, 원인은 무엇이며 어떻게 개선할 수 있을까?
- 특정 시점에서 Loss는 감소하지만 Accuracy가 오르지 않을 때 원인을 분석하세요.

Loss 감소와 **Accuracy** 상승 간의 불균형이 발생하는 이유는 **Loss**와 **Accuracy**의 측정 기준이 다르기 때문입니다.

Loss는 모델이 정답 클래스에 대해 얼마나 확신하는지를 연속적인 값으로 측정합니다. 반면 **Accuracy**는 최종 예측 클래스가 정답과 일치하는지만 판단하며, 모델이 정답 클래스의 확률을 높인다 하여도 **argmax**의 결과가 바뀌지 않으면 **accuracy**는 그대로 유지됩니다.

이러한 경우 **loss**는 감소하지만 **accuracy**는 정체되는 불균형이 발생할 수 있습니다.

불균형에는 다음과 같은 요인이 원인이 될 수 있습니다.

1. **learning rate**가 너무 작음
2. **optimizer**, 활성화 함수의 종류
3. **epoch**의 부족
4. 모델 구조 (은닉층 수, 뉴런 수 등등)

따라서, 이러한 불균형을 개선하기 위해선 **learning rate**를 조정하거나, **loss**가 계속해서 감소하는 상태라면 학습이 충분히 진행되지 않은 **under-training** 상태일 수 있으므로 **epoch**를 늘려 확인할 필요가 있습니다. **SGD**에서 정체가 심하게 발생하는 경우 **Adam**을 사용하거나 **Momentum**을 추가로 적용함으로써 해결할 수도 있으며, 또한 **Sigmoid**의 사용으로 인해 **gradient**가 작아지는 경우 **ReLU** 또는 **LeakyReLU** 활성화 함수로 변경하여 **gradient** 흐름을 개선하여금 해결할 수 있습니다.

- 3) Layer 별 Activation의 분포가 학습 중 어떻게 변화하는가?

- 초반, 중반, 후반 학습 시 활성화 함수 (ReLU, Sigmoid) 값의 분포를 시각화하고 분석하세요.

이론적으로 초기, 중기, 후기 학습 시 **ReLU**와 **Sigmoid**는 다음과 같은 형태의 **activation**이 발생해야 합니다.

ReLU

ReLU 함수의 식은 다음과 같습니다. (수식 : $\text{ReLU}(x) = \max(0, x)$)

초기 학습 단계에는 **ReLU**는 음수 입력을 0으로 차단합니다. 때문에 각 **layer**에서 0에 해당하는 **activation**이 일정 비율로 나타납니다. 특히 **layer**가 깊어질수록 이전 **layer**에서 0으로 제거된 정보의 영향을 받아 **activation sparsity**가 증가할 수 있습니다.

중기 학습 단계에서는 ReLU가 불필요한 입력을 0으로 제거하고, 분류에 필요한 특징에 대해서만 양수 activation을 생성합니다. 따라서 layer가 깊어질수록 activation 분포는 더 sparse해지지만, 일부 뉴런은 더 큰 activation 값을 가지며 특징 선택성이 강화됩니다.

후기 학습 단계에서는 ReLU activation 분포가 비교적 안정화되며, 깊은 layer에서는 분류에 중요한 특징만 선택적으로 활성화됩니다. 다만, 일부 뉴런이 지속적으로 0을 출력할 경우 Dead ReLU가 발생하여 해당 뉴런은 학습에 거의 기여하지 못하게 됩니다.

따라서 ReLU activation 을 사용할 경우, 레이어 별 이상적인 activation 분포는 아래와 같은 형태로 나타나야 합니다.

	초기	중기	후기
I1	0도 많고, 양수 activation도 적당히 존재	다양한 저수준 특징 활성화	넓은 범위의 기본 특징 반응
I2	Layer 1보다 0 비율 증가 가능	특정 패턴에 반응하는 뉴런 증가	중간 특징에 선택적으로 반응
I3	더 깊어질수록 0 activation 비율이 커질 수 있음	고수준 특징 위주로 강하게 활성화	최종 분류에 중요한 뉴런만 강하게 활성화

<이론상 발생해야 하는 ReLU activation 분포>

Sigmoid

Sigmoid 함수의 식은 다음과 같습니다. (수식 : $\text{Sigmoid}(x) = 1 / (1 + e^{(-x)})$)

초기 학습 단계에는 대부분의 뉴런이 애매하게 활성화된 상태로, weight가 랜덤이기에, activation이 보통 0.5 근처에 많이 몰려 있습니다.

반면, 중기 학습 단계에서는 학습이 진행되면서 일부 뉴런은 0이나 1에 가까운 값으로 이동합니다. 다만, Sigmoid는 0이나 1에 가까워질수록 gradient가 거의 0이 되기 때문에 포화 saturation 문제가 나타날 수 있는 문제가 있습니다.

ReLU activation 을 사용할 경우, 초기 및 중기의 레이어 별 이상적인 activation 분포는 아래와 같은 형태로 나타나야 합니다.

	초기	중기
I1	0.5 근처 값 많음	일부 뉴런이 0 또는 1 쪽으로 이동
I2	더 좁은 범위로 압축	activation 분포가 더 압축되거나 포화됨
I3	0.5 주변에 더 몰릴 수 있음	0/1 근처로 몰리면 gradient 감소

<이론상 발생해야 하는 Sigmoid activation 초기, 중기 레이어 분포>

후기 학습 단계에서는 **activation** 값이 0 또는 1에 가까운 영역에 머무를 수 있습니다. 이 경우 **gradient**가 거의 전달되지 않아 앞쪽 레이어의 학습이 느려지며, 이에 따라 **gradient vanishing** 문제가 발생할 수 있습니다.

	학습이 잘 된 경우	학습이 정체된 경우
I1	0 또는 1에 가까운 값으로 구분력 있게 분포	0.5 근처에 계속 몰림 또는 0/1에 포화되어 gradient 가 거의 흐르지 않음
I2		
I3		

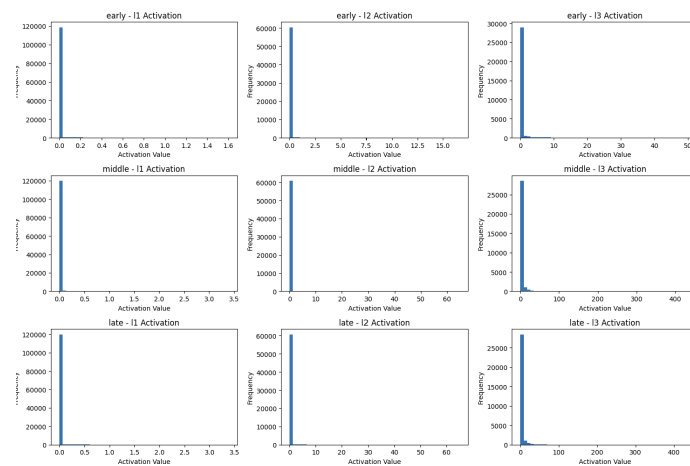
<이론상 발생해야 하는 Sigmoid activation 분포>

이러한 함수적 특성에 기인하여, 일반적으로 깊은 네트워크에서는 Sigmoid보다 ReLU 계열 활성화 함수가 더 안정적인 학습을 보장하는 경우가 많습니다.

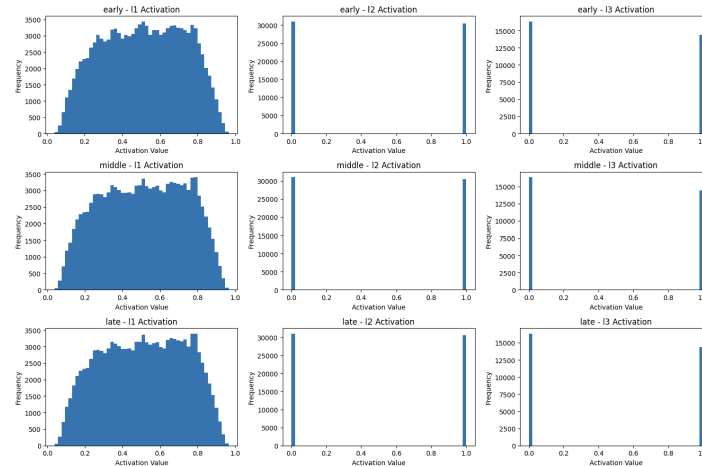
실험 결과

다만, 이번 B 실험에서는 학습 초기에 학습이 완료되는 형태였습니다.

때문에 초기, 중기, 후기 여부와 상관없이 모두 동일한 형태의, 학습 진행이 완료된 상태의 **activation**을 확인할 수 있었습니다.



<ReLU 학습별 layer 활성화 정도 : 위에서부터 초기, 중기 후기>



<sigmoid 학습별 layer 활성화 정도 : 위에서부터 초기, 중기 후기>

특히, ReLU의 경우 layer 1,2,3 모두 일부 뉴런만 activation된 반면, Sigmoid 함수를 활용한 경우 layer1의 뉴런은 대부분 활성화된 반면 layer 2,3의 뉴런은 일부만 activation 되었습니다.

learning rate를 내리는 등의 방법을 통해서 학습이 느리게 진행되도록 변형하여 재실험하면, 이러한 초기 중기 후기 학습의 이론적인 형태의 경향성을 시각적으로 확인할 수 있을 것이라 예상합니다.

4) Gradient Flow가 특정 층에서 소멸되거나 폭발할 때 모델에 미치는 영향은 무엇인가?

- Gradient Vanishing/Exploding이 발생한 구간을 찾아 원인을 분석하세요.

Gradient Flow는 각 레이어에 gradient가 얼마나 잘 전달되는지를 의미합니다. 학습은 gradient를 이용해 파라미터를 업데이트하는 과정이므로, gradient가 너무 작거나 너무 크면 학습이 정상적으로 진행되지 않을 수 있습니다.

Gradient Vanishing

Gradient Vanishing은 뒤쪽 layer에서 앞쪽 layer로 gradient가 전달되는 과정에서 값이 점점 작아져, 앞쪽 layer가 거의 업데이트되지 않는 현상입니다. 실험B#3에서 Sigmoid 모델을 사용하여 진행한 실험의 경우, 특정 시점 이후 loss와 accuracy가 거의 변하지 않는 학습 정체 구간을 보였습니다.

이러한 현상은 Sigmoid activation이 0 또는 1 근처로 포화되면서 gradient가 매우 작아졌기 때문으로 해석할 수 있습니다.

Exploding Gradient

Exploding Gradient는 gradient 값이 지나치게 커져 파라미터 업데이트가 과도하게 발생하는 현상입니다. 이 경우 loss가 안정적으로 감소하지 않고 크게 튀거나, accuracy가 불안정하게 진동할 수 있습니다.

실험C#2와 같은 SGD+Momentum 활성화 함수에서 큰 **learning rate**를 이용해 실험을 진행하는 경우, 이론적으로는 이전 업데이트 방향을 누적하여금 최적점을 지나쳐 **overshooting**이 발생할 수 있습니다.

다만, 이번 실험 결과에서는 **learning rate**가 0.1으로 큰 경우에도 **loss**가 감소하는 경향을 보이고 있으므로 **exploding gradient**가 발생하는 모습은 확인할 수 없었습니다.

The end of page